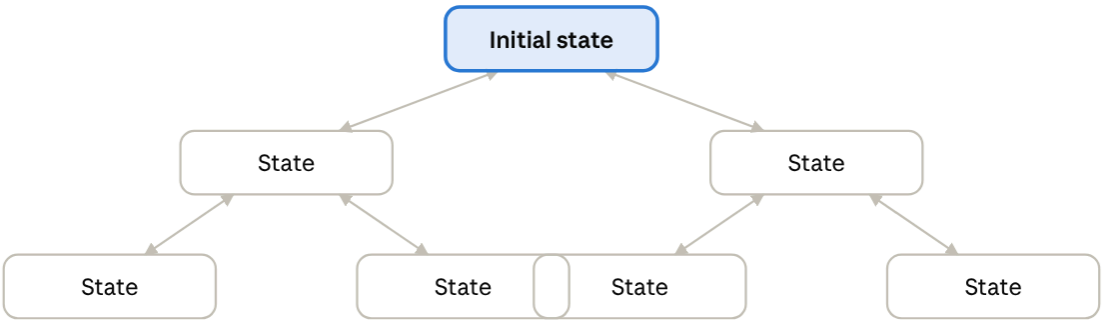
Search Artificial Intelligence

Oct 20, 2026 · @Durga prasad Avala

Terminology

- **Agent:** an entity that perceives its environment and acts upon that environment.
  - *Perceives:* to become aware of, notice, or understand something through your physical senses (sight, sound, smell, touch, taste) or your mind.
- **State:** a configuration of the agent and its environment.
  - *Configuration:* the way something is set up or arranged so that it works in a particular way. Basically, how something is set up to work.
- **Initial state:** the state in which the agent begins.
- **Actions:** choices that can be made in a state.
  - Actions(s) returns the set of actions that can be executed in state s.
- **Transition model:** a description of what state results from performing any applicable action in any state.
  - Result(s, a) returns the state resulting from performing action a in state s.
  - Example: Result(a board with some squares filled, move a tile into the empty square) returns the board as it looks after that move.
- **State space:** the set of all states reachable from the initial state by any sequence of actions.
  - Drawn as a start state with arrows to the states one action away, and from those to the states one further action away.
- **Abstract representation:** a state space drawn as circles (states) joined by arrows (actions).
  - *Abstract:* something that is not physical or concrete, or something that expresses an idea rather than showing a specific real-world object.
- **Goal test:** a way to determine whether a given state is a goal state.
- **Path cost:** a numerical cost associated with a given path.
  - Path cost is used to find a path with optimal (lowest) cost.
  - In the example graph the edge costs differ, so the cost of a path is the sum of its edges (costs range from 1 up to 9).
  - For the 15-puzzle (15 boxes) every move costs the same (equal cost).

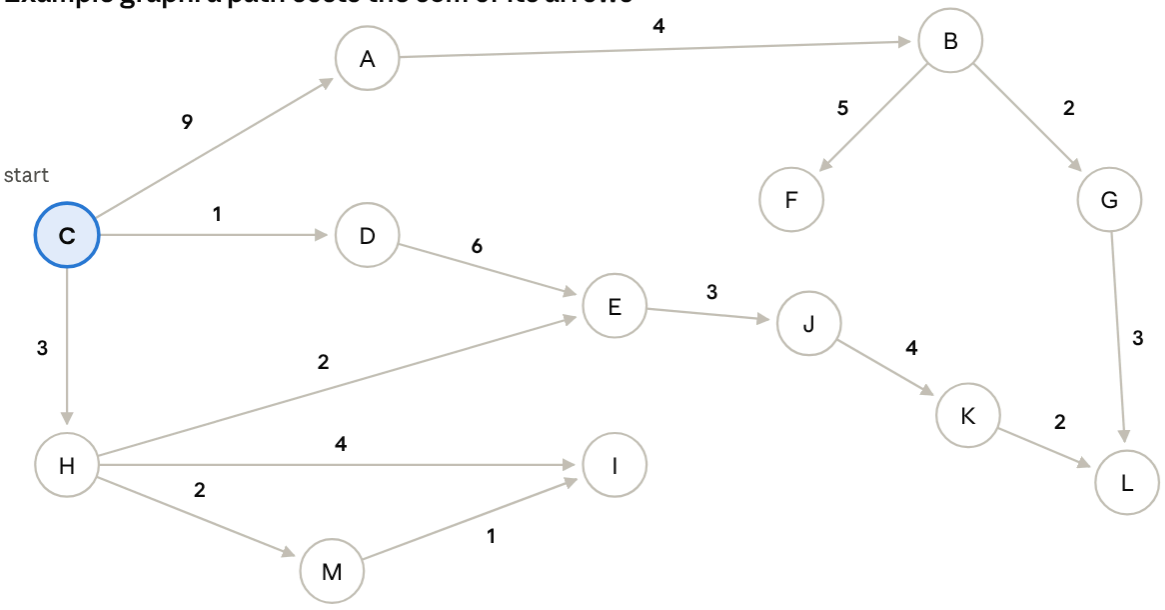
State space: every state reachable from the initial state



state space · initial state and the states one and two actions away

Each arrow is an action, and the arrows run both ways because an action can lead back to an earlier state.

Example graph: a path costs the sum of its arrows



weighted graph · 13 nodes, 15 arrows with costs

Example for path cost: each number is the cost of that arrow.

Search Problems

A search problem has five parts:

- Initial state
- Actions
- Transition model
- Goal test

- Path cost function

Solution: a sequence of actions that leads from the initial state to the goal state.

Optimal solution: a solution that has the lowest path cost among all solutions.

Node: a data structure that keeps track of:

- a state
- a parent (the node that generated this node)
- an action (the action applied to the parent to get this node)
- a path cost (from the initial state to this node)

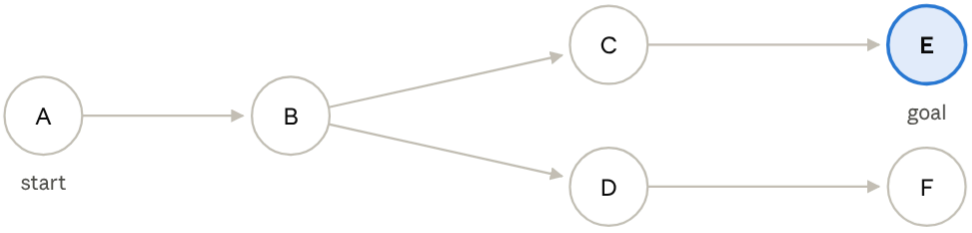
Approach

Start with a frontier that contains the initial state. Then repeat:

- If the frontier is empty, there is no solution.
- Remove a node from the frontier.
- If the node contains the goal state, return the solution.
- Expand the node and add the resulting nodes to the frontier.

Example: find a path from A to E. The graph is A to B, B to C and D, C to E, D to F.

Example graph: A leads to B, B to C and D, C to E, D to F



example graph · 6 nodes, goal E

The same graph is used for the stack and queue examples below.

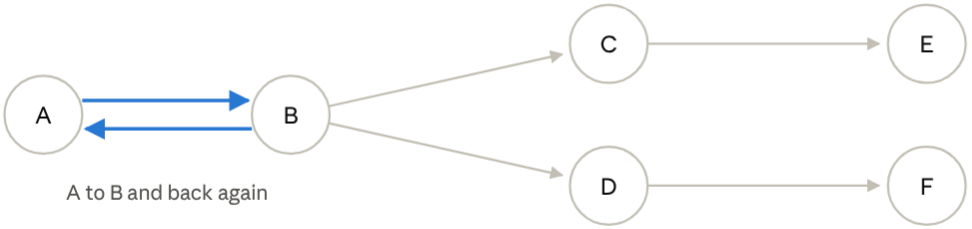
1. Frontier: A. Remove A (not the goal), expand it, add B.
2. Frontier: B. Remove B (is B the goal? No), expand it, add C and D.
3. Frontier: C, D. Remove C, expand it, add E.
4. Frontier: D, E. Remove E.
5. Is E the goal state? Yes, so return the solution: the path A, B, C, E.

The order in which nodes are removed from the frontier is not fixed yet; the stack and queue sections below decide it.

Fragile Process

The first approach fails when the graph has a two-way link. Suppose A and B are joined in both directions, B leads to C and D, C leads to E, and D leads to F.

A and B link both ways, so the search can loop between them forever



graph with a two-way A–B link

The highlighted two-way link is what sends the search round in circles.

1. You have your frontier and you add A to it.
2. You remove A and go to B.
3. You remove B and try to go to the next states, which will be A, C and D.
4. Say you take out A again: it gives you B. Then B again gives you C and D.
5. This is like an infinite loop, so our approach fails.

Revised Approach and Depth-First Search

Start with a frontier that contains the initial state, and start with an empty explored set. Then repeat:

- If the frontier is empty, there is no solution.
- Remove a node from the frontier.
- If the node contains the goal state, return the solution.
- Add the node to the explored set.
- Expand the node and add the resulting nodes to the frontier if they aren't already in the frontier or the explored set.

In what order do we remove nodes from the frontier? The first option is a **stack**, a last-in, first-out data type.

Example (A to B, B to C and D, C to E, D to F; the goal is E), using a stack:

1. Add A to the frontier. Remove A: is it the goal? No. Add A to the explored set.
2. Next node: B. Remove B: not the goal, so add it to the explored set. Add C and D to the frontier.

3. Which to remove first? Apply the stack (last-in, first-out): remove D. Is it the goal? No. Add it to the explored set.
4. Add the next node from D, which is F. Is it the goal? No. Add it to the explored set.
5. Frontier: C. Not the goal, so add it to the explored set. Next node: E.
6. Is E the goal state? Yes.

Answer: E, found by using the stack removal approach. Explored set: A, B, D, F, C.

This type of search is called **depth-first search**: a search algorithm that always expands the deepest node in the frontier.

Breadth-First Search

Is there another algorithm? Yes: **breadth-first search**, a search algorithm that always expands the shallowest node in the frontier.

In depth-first search we used a stack, so we went deep into the search along the node we chose by the stack method. In breadth-first search we use a **queue** (first-in, first-out), which means we pick the node that came first into the frontier, not the one that came last (which is what a stack does).

Example (same graph: A to B, B to C and D, C to E, D to F; the goal is E):

1. Add A. Is it the goal? No. Remove it, add it to the explored set, and add the next node.
2. Add B. Is it the goal? No. Add it to the explored set. For the next nodes, check against the explored set and add only those that are not in it.
3. Add C and D. Using the queue method, remove C: is it the goal? No. Add it to the explored set and add the next node from C, which is E.
4. Frontier: D, E (D is at the front of the queue). Remove D: not the goal, so add it to the explored set.
5. Remove E. Is it the goal state? Yes. Return the solution.

Explored set: A, B, C, D.

Suppose there are two paths to the goal state. Breadth-first search takes both paths simultaneously, one step at a time, until it gets to the goal state. The drawback is that you might meet a situation where breadth-first search uses a lot of memory to find the goal state.

Uninformed and Informed Search

Assume we are in a maze, we start from A, and our goal is to get to B. We encounter a situation where we have two paths. A human would look at the path that will take them to the goal state and choose that path. Can an algorithm do this as well? Yes, but we have to divide this into two methods: informed and uninformed search.

- **Uninformed search:** a search strategy that uses no problem-specific knowledge (for example, BFS and DFS).
- **Informed search:** a search strategy that uses problem-specific knowledge to find the solution more efficiently.

Types of Informed Search: Greedy Best-First Search

Greedy best-first search is a search algorithm that expands the node that is closest to the goal, as estimated by a heuristic function h(n).

The search algorithm isn't going to know for sure whether a node is the closest thing to the goal. If we knew which node is closest to the goal, we would already have the solution. Instead, we can use an estimate of what's closest to the goal, otherwise known as the **heuristic function**.

Heuristic function. It answers which of two options (for example C or D) would be better if you are trying to find a way to your goal. A human would say D is the closest, because anyone can see it is closest to B (the goal). If you know the (x, y) coordinates of C, D and the goal, you will know which is closer. This is what we use the heuristic function for.

Manhattan distance. One specific type of heuristic: how many squares you must move vertically and horizontally to reach the goal. You are not allowed to go diagonally, only up, down, left or right. To find the goal state, we label each cell with how far away it is from the goal.

Maze: every cell shows how many blocks it is from the goal B

Manhattan distance: only up, down, left or right, no diagonals. Shaded cells are walls.

	10	9	8	7	6	5	4	3	2	1	**B**
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

A = start B = goal, top right

maze · each cell labelled with its blocks to B

The maze is labelled this way, so the search can compare neighbouring cells.

Approach (maze with A as the start and B as the goal, each cell labelled with its Manhattan distance to B):

1. Start at A and go to 14.
2. Check which side is closest to the goal, which is the 13 cell.
3. Take the 13 cell, then go on to the cell labelled 8. Both sides are 8, so we make an arbitrary choice.
4. At an 8 cell (D) we have two choices, 7 or 9. Since 7 < 9 we choose 7, and then we are at 6.
5. At 6 we have two choices, 5 or 7. Since 5 < 7 we choose 5. We go until 2, where there is no path back to 7.
6. So we follow the 7 instead, and we reach B (the goal state).

Exception. The greedy choice can be wrong: the cell with the greater value at that moment might seem to be a longer path, but sometimes it is the best path as well. The heuristic value went low by choosing the smaller cell at that moment, but it goes up again in the process of finding the goal state. To solve this we use A* search.

A* Search

A* search is a search algorithm that expands the node with the lowest value of g(n) + h(n).

- g(n): the cost to reach the node (how far did I have to travel to get here?).

- h(n): the estimated cost to the goal (how far away am I from the goal?).

A* search with Manhattan distance labels. In the maze, each cell is written as two numbers: the Manhattan distance to the goal B (h) and the number of steps travelled from A (g). Add them to get g(n) + h(n).

A* maze: each cell reads blocks to B + steps from A

For example 11+6 means 11 blocks from B and 6 steps taken. Shaded cells are walls.

	10+11	9+12	8+13	7+14	6+15	5+16	4+17	3+18	2+19	1+20	**B**
	11+10										1
	12+9		10+7	9+8	8+9	7+10	6+11	5+12	4+13		2
	13+8		11+6						5+14		3
	14+7	13+6	12+5		10	9	8	7	6+15		4
			13+4		11						5
A	16+1	15+2	14+3		12	11	10	9	8	7	6

Highlighted: the route over the top, 21 steps from A to B

One number = blocks to B only

A* maze · cells labelled blocks to B + steps from A

At the fork after 12 + 5, the cell 11 + 6 (17 in total) beats 13 + 6 (19 in total), so A* takes it first.

Worked comparison:

- Between 11 + 6 and 13 + 6, we choose 11 + 6 since it takes fewer steps.
- 6 + 15: I have travelled 15 steps and the estimated distance to the goal is 6, so 15 + 6 = 21.
- Or I could be at 6 + 13, which is 19.
- Since 19 < 21, I would rather take 19 (fewer steps) over 21 (more steps) to reach the goal. Maybe there is a better path by 17 + 6 (17 blocks away and 6 steps taken).

A* is optimal if:

- h(n) is **admissible**: it never overestimates the true cost (it should never assume a node is further away from the goal than it really is).
- h(n) is **consistent**: for every node n and successor n′ with step cost c, $h(n) \leqslant h(n') + c$.

Adversarial Search and Minimax

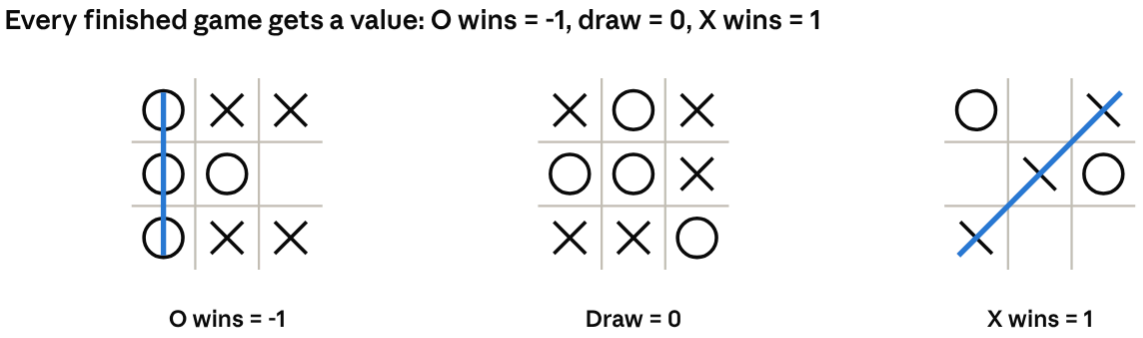
Adversarial search is where an agent is trying to make intelligent decisions, and there is someone (another agent or a human) fighting against it. You are trying to succeed and someone wants you to fail. Example: tic-tac-toe.

To deal with this type of adversarial situation we need an algorithm called **Minimax**.

Minimax works very well when there are two players. I make a move, my opponent makes a move, and I am trying to win and my opponent is also trying to win.

What do we need to make this algorithm work? Anytime we translate the human concept of playing a game, winning and losing, to a computer, we want to express it in terms the computer can understand. Ultimately a computer only understands numbers, so we want some way of translating a game of X's and O's on a grid into something numerical.

What we do is take each possible way a tic-tac-toe game can unfold and assign a value, or **utility**, to each one of those possible ways. This works not just for tic-tac-toe but for any game.



tic-tac-toe end states · values -1, 0, 1

- Assign each possibility a value from -1, 0 and 1.
- **Max (X)** aims to maximize the score.
- **Min (O)** aims to minimize the score.

Game Definition for Minimax

A game is described by these parts:

- **S0:** the initial state.
- **Player(s):** returns which player is to move in state s.
- **Actions(s):** returns the legal moves in state s.
- **Result(s, a):** returns the state after action a is taken in state s (the transition model).

- **Terminal(s):** checks whether state s is a terminal state, for example a goal state, someone has won, or no more moves can be made (tic-tac-toe).
- **Utility(s):** the final numerical value for terminal state s.

Initial state. In tic-tac-toe this is the empty board.

Player(s) (suppose X makes the first move):

- Player(empty board) is X, who makes the move.
- Player(board with one X on it) is O, who makes the move.

Actions(s) takes a state and gives all the possible actions of that state. Example: X is in the top-middle, middle, middle-right and bottom-left squares, O is in the top-right, middle-left and bottom-right squares, and it is O's turn. Two squares are empty, so Actions(s) is the set of two boards: O in the top-left, or O in the bottom-middle.

Result(s, a) example: Result(that board, O in the top-left) returns the board with an O placed in the top-left square.

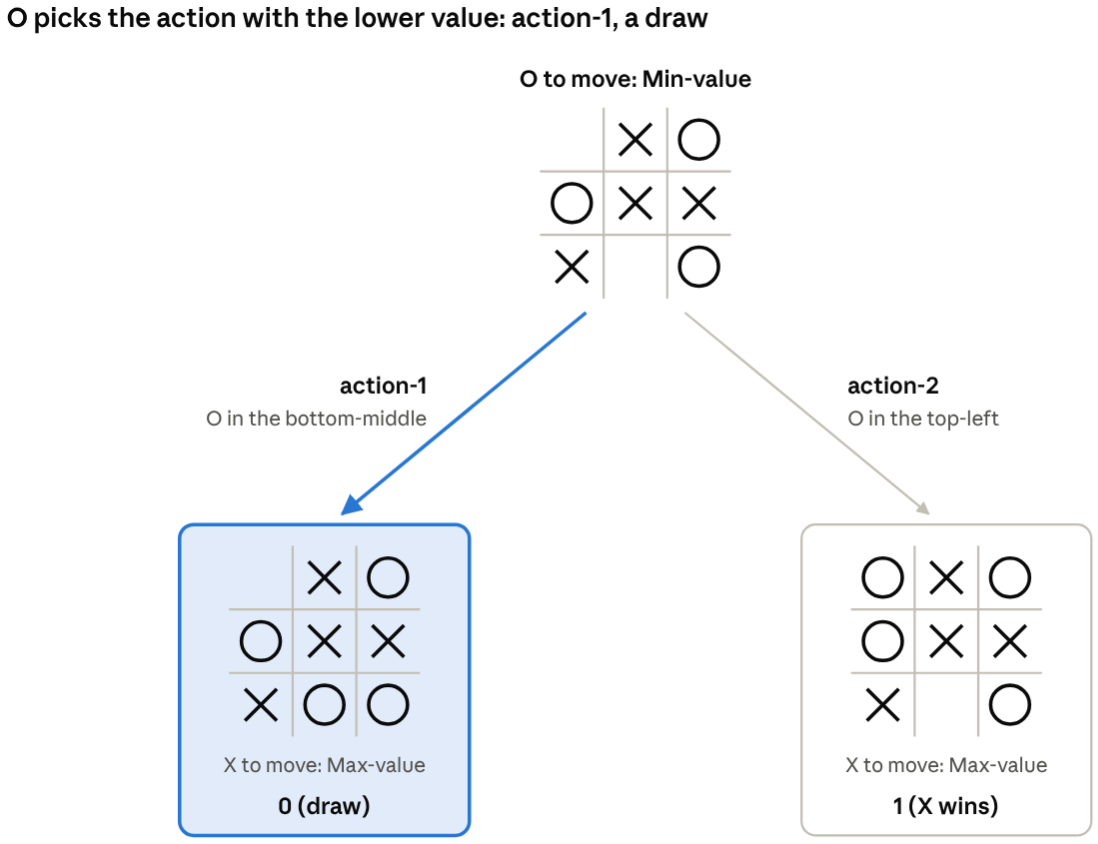
Terminal(s) takes a state and tells you if it is over:

- an unfinished board returns false (not over)
- a board where someone has won returns true (over)
- a full board returns true (over)

Utility(s): when the game is over, what is the value of the game? X wins gives 1, O wins gives -1, and a draw gives 0. For example U(a board X has won) is 1, U(a board O has won) is -1, and U(a full board with no winner) is 0.

Minimax Worked Example

Player(s) is O, who makes the move, on the board used above (X in the top-middle, middle, middle-right and bottom-left squares; O in the top-right, middle-left and bottom-right squares).



Min-value example · O chooses between two actions

O's action depends on the score. Since O is trying to bring the score down, it checks which action helps that and performs it. At the same time it is trying to win against X, so it takes the action that also won't let X win. This is the Min-value step: O is trying to minimize the value, since X is trying to maximize it.

- **Action-1** (O in the bottom-middle): X then plays the last square, the top-left, and the game is a draw. Value 0.
- **Action-2** (O in the top-left): X then plays the bottom-middle and completes the middle column. Value 1 (X wins).

Since O can't win and won't let X win, it goes with action-1.

The Minimax Algorithm

Generic game, abstract view. An up triangle is Max and a down triangle is Min.

generic game, abstract view · two small Minimax trees

- Top tree: the Max node sees three Min choices worth 5, 3 and 9. It looks for the choice that gives it the highest value, which is 9, so it chooses that.
- Bottom tree (the opponent): the left Max node takes the best of 5, 3 and 9, which is 9. The right Max node takes the best of 2 and 8, which is 8. The Min node at the top then picks the smaller of 9 and 8, which is 8.

Minimax. Given a state s:

- Max picks an action a in Actions(s) that produces the highest value of Min-Value(Result(s, a)).
- Min picks an action a in Actions(s) that produces the smallest value of Max-Value(Result(s, a)).

```
function Max-Value(State):
    if Terminal(State):
        return Utility(State)
    v = -infinity
    for action in Actions(State):
        v = Max(v, Min-Value(Result(State, action)))
    return v
```

Min-Value is the mirror image of this function: it starts with v = +infinity, keeps the smaller value, and uses Max-Value of each result.

Alpha-Beta Pruning

Alpha-beta pruning is a way to make Minimax faster. Plain Minimax looks at every branch of the game tree, which gets expensive. Alpha-beta pruning skips branches that cannot change the final decision.

The idea: while searching, keep two bounds.

- **Alpha:** the best (highest) value Max is already guaranteed on the path so far.
- **Beta:** the best (lowest) value Min is already guaranteed on the path so far.

Once a Min node finds a value that is no better for Max than a value Max already has elsewhere, Max will never choose that node, so its remaining children are skipped. The same works in mirror for Max nodes. Stop looking at a node as soon as alpha is greater than or equal to beta.

alpha-beta pruning example · 9 leaves, 3 pruned

Example. The root is a Max node with three Min children, and the leaves are the values 4, 5, 9 | 9, 3 | 2.

1. The first Min node looks at 4, 5 and 9. Min takes the smallest, so its value is 4. Max now knows it can get at least 4.
2. The second Min node looks at 9, then 3. Its value is at most 3 (written ⩽3), which is already worse for Max than 4. Max will never pick it, so its third child is never looked at.
3. The third Min node looks at 2. Its value is at most 2 (written ⩽2), also worse than 4, so its other two children are skipped.
4. Max picks the largest of 4, ⩽3 and ⩽2, which is 4.

This is the same answer plain Minimax would give, but only 6 of the 9 leaves were examined.